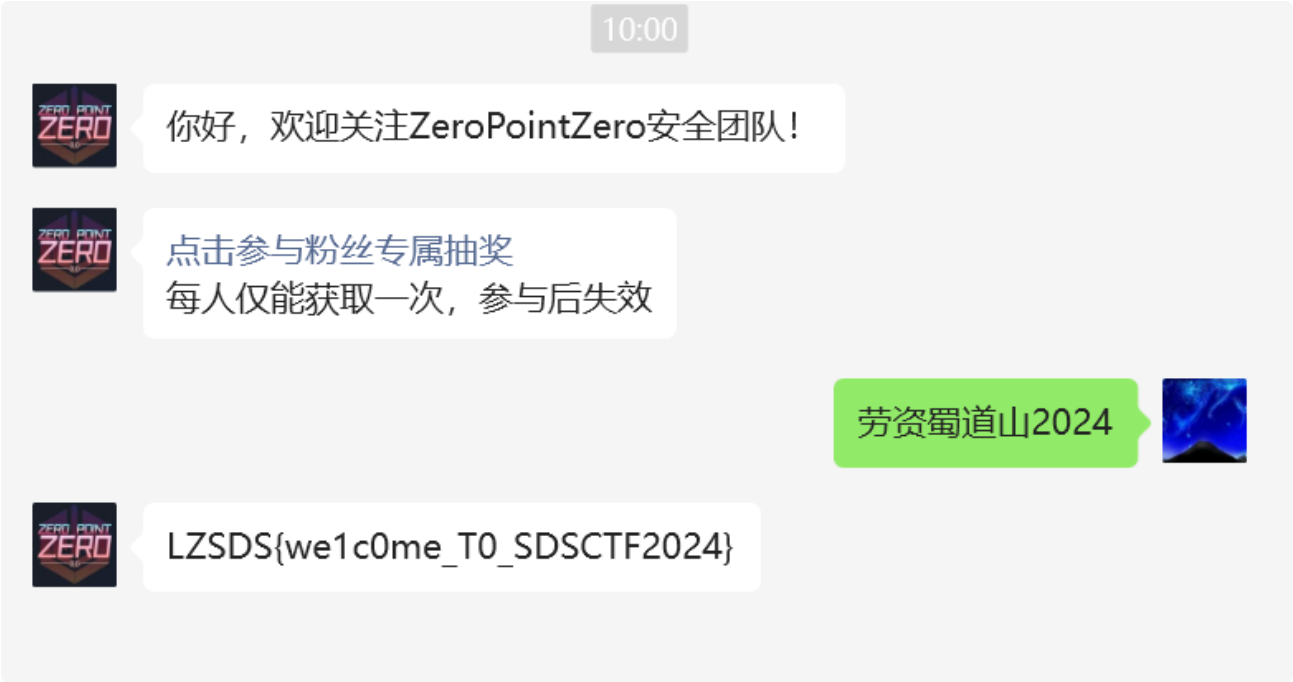
蜀道山高校公益联赛

MISC

欢迎来到2024蜀道山CTF



神奇的硬币纺纱机

```
[i] 诶，对方投币了，你成功拿到了 90 枚硬币！(尚余 314)
[?] (314@94) 投币吗 <10?i>: 0
[i] 诶，对方投币了，你成功拿到了 90 枚硬币！(尚余 404)
[?] (404@95) 投币吗 <10?i>: 0
[i] 这次你俩都没投币哦。(尚余 404)
[?] (404@96) 投币吗 <10?i>: 0
[i] 这次你俩都没投币哦。(尚余 404)
[?] (404@97) 投币吗 <10?i>: 0
[i] 这次你俩都没投币哦。(尚余 404)
[?] (404@98) 投币吗 <10?i>: 0
[i] 这次你俩都没投币哦。(尚余 404)
[i] 你愿意为了 flag，尽全力一搏吗？
[i] 记住这个数字：8 / 91
[?] (404@99) 投币吗 <10?i>: 0
[i] 诶，对方投币了，你成功拿到了 1000 枚硬币！(尚余 1404)
[i] 恭喜你过关了！
[i] 你最终获得了 1404 枚硬币。
[i] 这是你的 flag: flag{ef39f06fd2161cfe3a134d599560fd78}
```

怎么一直按 0 都能出 flag了

Elemental Wars

这回蒙不出来了，只能和它爆了

```python
from pwn import *
import random

context(os='linux', log_level='debug')
conn = remote('gamebox.yunyansec.com', 15924)

for _ in range(500):
    conn.recvuntil(': ')
    # conn.sendline(b'1')
    num = random.randint(1, 5)
    conn.sendline(str(num).encode())

conn.interactive()
```

```
000000b0  0a e4 bd a0  e8 8e b7 e5  be 97 e4 ba  86 e8 83 9c  ····  ····  ····  ····
000000c0  e5 88 a9 ef  bc 81 e6 95  8c e4 ba ba  e5 b7 b2 e8  ····  ····  ····  ····
000000d0  a2 ab e5 87  bb e8 b4 a5  ef bc 81 0a  e6 ac a2 e8  ····  ····  ····  ····
000000e0  bf 8e e6 8b  bf e5 88 b0  66 6c 61 67  3a 66 6c 61  ····  ····  flag  :fla
000000f0  67 7b 30 35  61 35 66 30  62 62 66 36  61 33 30 63  g{05  a5f0  bbf6  a30c
00000100  36 32 36 32  38 30 32 31  38 32 61 63  63 61 62 36  6262  8021  82ac  cab6
00000110  32 39 7d 0a  0a e8 af b7  e8 be 93 e5  85 a5 e9 80  29}·  ····  ····  ····
00000120  89 e9 a1 b9  28 31 3a 20  e7 bb a7 e7  bb ad e6 b8  ····  (1:   ····  ····
00000130  b8 e6 88 8f  2c 20 32 3a  20 e6 9f a5  e7 9c 8b e8  ····  , 2:   ···  ····
00000140  a7 84 e5 88  99 2c 20 33  3a 20 e9 80  80 e5 87 ba  ····  ·, 3  : ··  ····
00000150  29 3a 20 e6  84 9f e8 b0  a2 e6 b8 b8  e7 8e a9 ef  ):  ·  ····  ····  ····
00000160  bc 81 e5 86  8d e8 a7 81  ef bc 81 0a              ····  ····  ····
0000016c
择你的元素（1. 金，2. 木，3. 水，4. 火，5. 土）：
你的选择是：土
敌人的选择是：水
土克水，玩家获胜！
你的血量：13，敌人的血量：0
你获得了胜利！敌人已被击败！
欢迎拿到flag:flag{05a5f0bbf6a30c6262802182accab629}

请输入选项(1: 继续游戏，2: 查看规则，3: 退出)：感谢游玩！再见！
[*] Got EOF while reading in interactive
$
[*] Interrupted
[*] Closed connection to gamebox.yunyansec.com port 15924
```

javaPcap

流量包有个好的一点就是 cmd 参数可以直接 base64 解。三个加密我只需要看 sm4 和 blowfish。

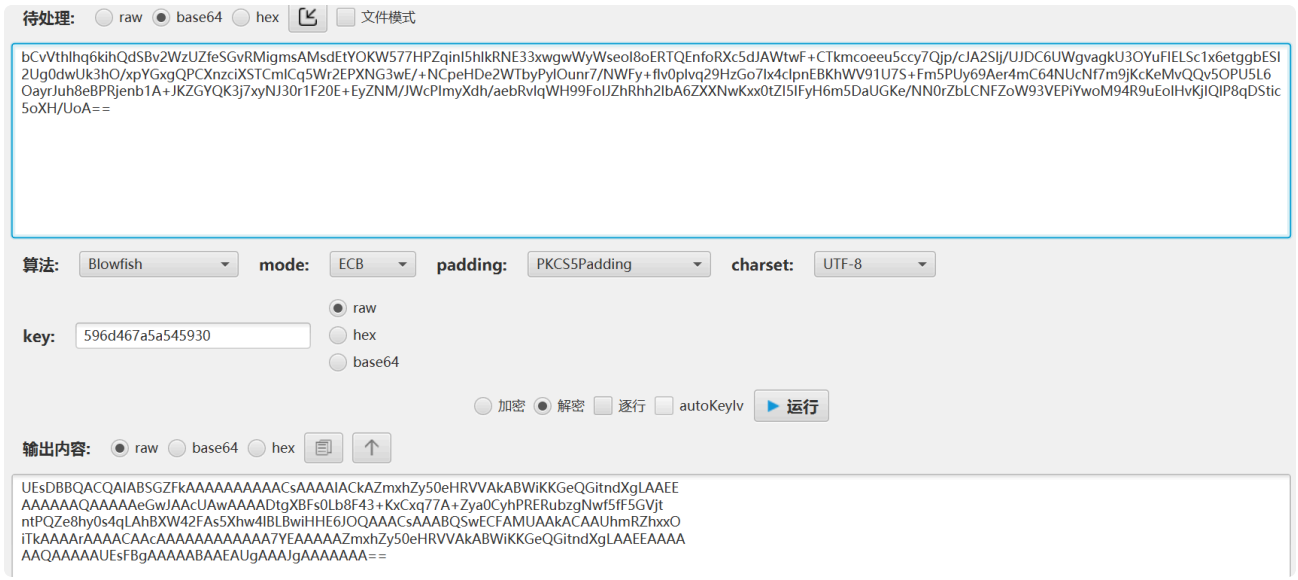
```java
public class ShellHandler implements HttpHandler {
    public void handle(HttpExchange exchange) throws IOException {
        String requestParam = exchange.getRequestURI().getQuery();
        Map<String, String> params = parseQueryParams(requestParam);
        String encryptMethod = params.get("encryptMethod");
        String key = params.get("key");
        String cmd = new String(Base64.getDecoder().decode(params.get("cmd")));
        try {
            String execContent = ExecCmd(cmd);
            System.out.println(execContent);
            String encryptContent = "null";
            boolean z = -1;
            switch (encryptMethod.hashCode()) {
                case -646251254:
                    if (encryptMethod.equals("blowfish")) {
                        z = 2;
                        break;
                    }
                    break;
                case 96463:
                    if (encryptMethod.equals("aes")) {
                        z = false;
                        break;
                    }
                    break;
                case 113946:
                    if (encryptMethod.equals("sm4")) {
                        z = true;
                        break;
                    }
                    break;
            }
```

顺着 shell 往下走就可以看到执行了 `base64 flag/flag.zip`

```
GET /shell?cmd=YmFzZTY0IGZsYWcvZmxhZy56aXA&key=596d467a5a5459304947 5a73595763765a6d78685a793536615841&encryptMethod=blowfish HTTP/1.1
Host: 139.199.77.49:8080
User-Agent: python-requests/2.31.0
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive


HTTP/1.1 200 OK
Date: Wed, 06 Nov 2024 07:30:37 GMT
Content-length: 472

bCvVthlhq6kihQdSBv2WzUZfeSGvRMigmsAMsdEtYOKW577HPZqinI5hlkRNE33xwgwWyWseoI8oERTQEnfoRXc5dJAWtwF+CTkmcoeeu5ccy7Qjp/cJA2Slj/UJDC6UWgvagkU3OYuFIELSc1x6etggbE
Sl2Ug0dwUk3hO/xpYGxgQPCXnzciXSTCmlCq5Wr2EPXNG3wE/+NCpeHDe2WTbyPylOunr7/NWFy+flv0plvq29HzGo7lx4clpnEBKhWV91U7S+Fm5PUy69Aer4mC64NUcNf7m9jKcKeMvQQv5OPU5L6Oay
rJuh8eBPRjenb1A+JKZGYQK3j7xyNJ30r1F20E+EyZNM/JWcPImyXdh/aebRvlqWH99FoIJZhRhh2lbA6ZXXNwKxx0tZI5IFyH6m5DaUGKe/NN0rZbLCNFZoW93VEPiYwoM94R9uEolHvKjIQlP8qDStic
5oXH/UoA==
```

Blowfish 解密，key 是前 16 个字符： `596d467a5a545930`

```java
public static String encryptBlowfish(String key, String content) {
    return encrypt(content, key, "Blowfish", 16);
}

private static String encrypt(String content, String key, String algorithm, int keySize) {
    try {
        byte[] keyBytes = new byte[keySize];
        byte[] inputKeyBytes = key.getBytes(StandardCharsets.UTF_8);
        System.arraycopy(inputKeyBytes, 0, keyBytes, 0, Math.min(inputKeyBytes.length, keySize));
        SecretKeySpec secretKey = new SecretKeySpec(keyBytes, algorithm);
        Cipher cipher = Cipher.getInstance(algorithm + "/ECB/PKCS5Padding");
        cipher.init(1, secretKey);
        byte[] encryptedBytes = cipher.doFinal(content.getBytes(StandardCharsets.UTF_8));
        return Base64.getEncoder().encodeToString(encryptedBytes);
    } catch (Exception e) {
        throw new RuntimeException("Encryption error: " + e.getMessage(), e);
    }
}
```



解密完就是 base64 过后的压缩包，压缩包还有密码，接着往下看有 `cat flag/hint.txt`，解密 SM4

```
GET /shell?cmd=Y2F0IGZsYWcvaGludC50eHQ&key=5932463049475a735957637661476c7564433530654851&encryptMethod=sm4 HTTP/1.1
Host: 139.199.77.49:8080
User-Agent: python-requests/2.31.0
Accept-Encoding: gzip, deflate
Accept: */*
Connection: keep-alive


HTTP/1.1 200 OK
Date: Wed, 06 Nov 2024 07:30:39 GMT
Content-length: 192

/RrfjvgK6Kh8HS3P7KcDScgyhFsidoqnnPhO5y7O6r8d4XrgJ3Fiph6ZF9jwOcmNc9Hd5/mH+ciddkXl4eCftq2RrHZyhAf178Gelz73yzJb0/Is8s1vEWQQ/bcrEyTWJ1+qk+D6pmri/wJkLxfYAJQBFP
63eVURRV2ki9j6qLMcDaNJBEh0PjKaSU+EKGI9
```

```java
public static String encryptSM4(String key, String content) {
    try {
        byte[] keyBytes = generateKeyBytes(key, 16);
        SecretKeySpec secretKey = new SecretKeySpec(keyBytes, "SM4");
        Cipher cipher = Cipher.getInstance("SM4/ECB/PKCS5Padding", "BC");
        cipher.init(1, secretKey);
        byte[] encryptedBytes = cipher.doFinal(content.getBytes(StandardCharsets.UTF_8));
        return Base64.getEncoder().encodeToString(encryptedBytes);
    } catch (Exception e) {
        throw new RuntimeException("Encryption error: " + e.getMessage(), e);
    }
}
```

密码为执行命令（按照时间排序）的首字母的组合重复三次，比如执行了（id,whoami），那么密码就为iwiwiw

一共执行了 5 条命令

```
whoami
ls -alt
ls flag/
base64 flag/flag.zip
cat flag/hint.txt
```

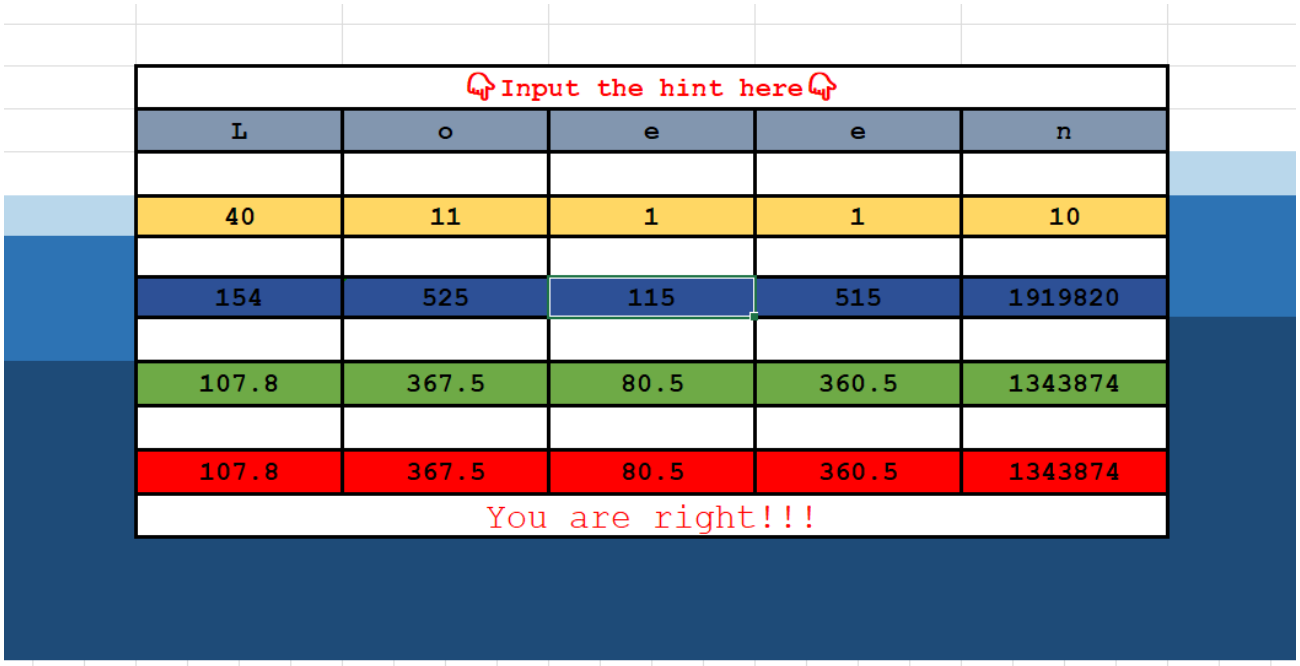
密码就是wllbcwllbcwllbc

`LZSDS{java_also_can_chaozuoPCAPNG_LUCKFORU}`

Summit Potato

Hyaya
vmare

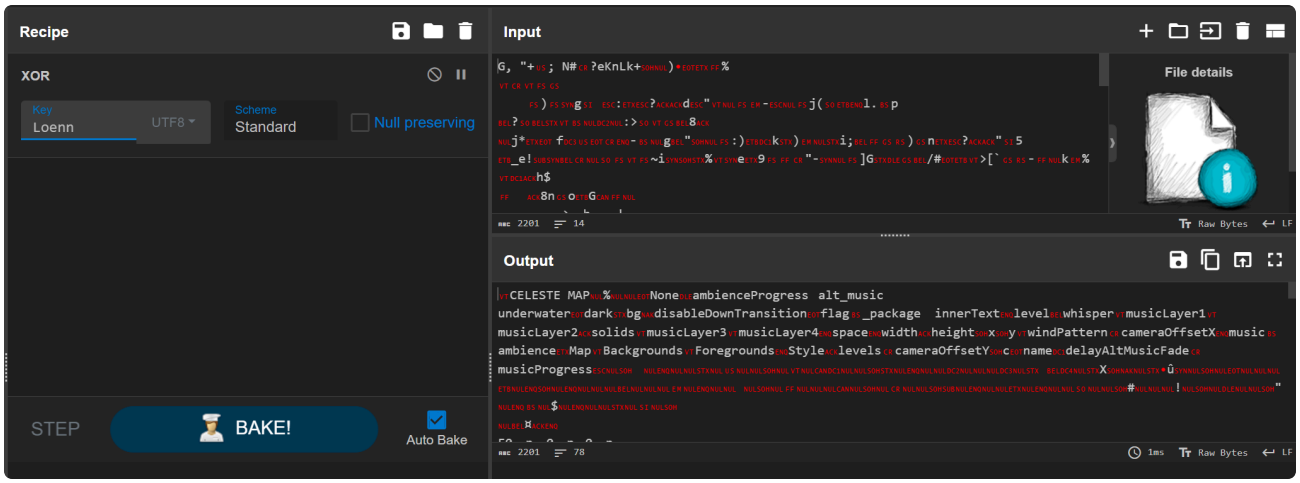
先分出来个 xlsx，怎么还是 xlsx 计算器



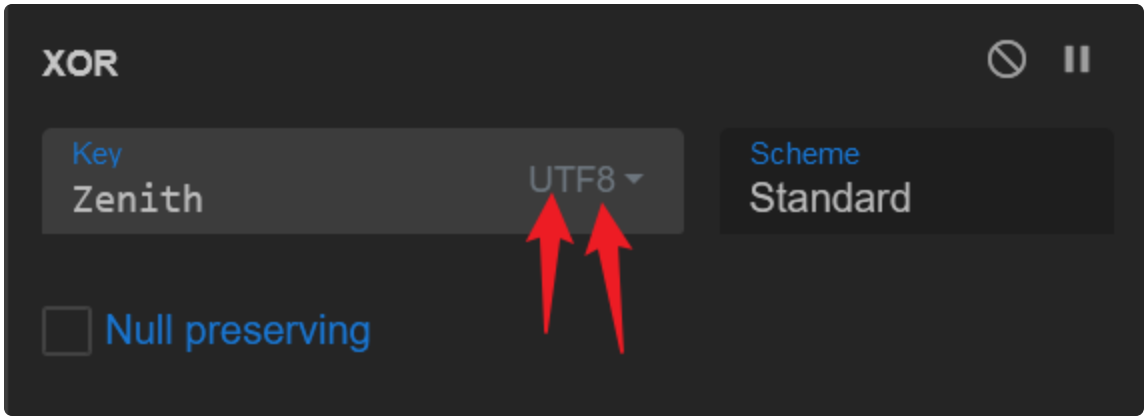
Input the hint here				
L	o	e	e	n
40	11	1	1	10
154	525	115	515	1919820
107.8	367.5	80.5	360.5	1343874
107.8	367.5	80.5	360.5	1343874
You are right!!!				

```python
a = [107.8, 367.5, 80.5, 360.5, 1343874]

for i in range(len(a)):
    a[i] = int(a[i] / 0.7)
print(a)

a[0] = a[0] - 114
a[1] = a[1] - 514
a[2] = a[2] - 114
a[3] = a[3] - 514
a[4] = a[4] - 1919810
print(a)

for i in range(len(a)):
    a[i] = chr(a[i] ^ 100)
print(a)

"""Loeen"""
```

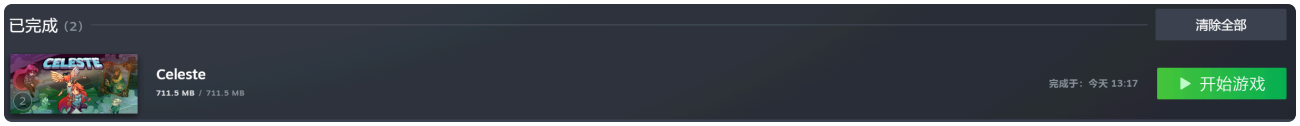
结果这里得出来 `Loeen`，实际异或还是试出来的 `Loenn`



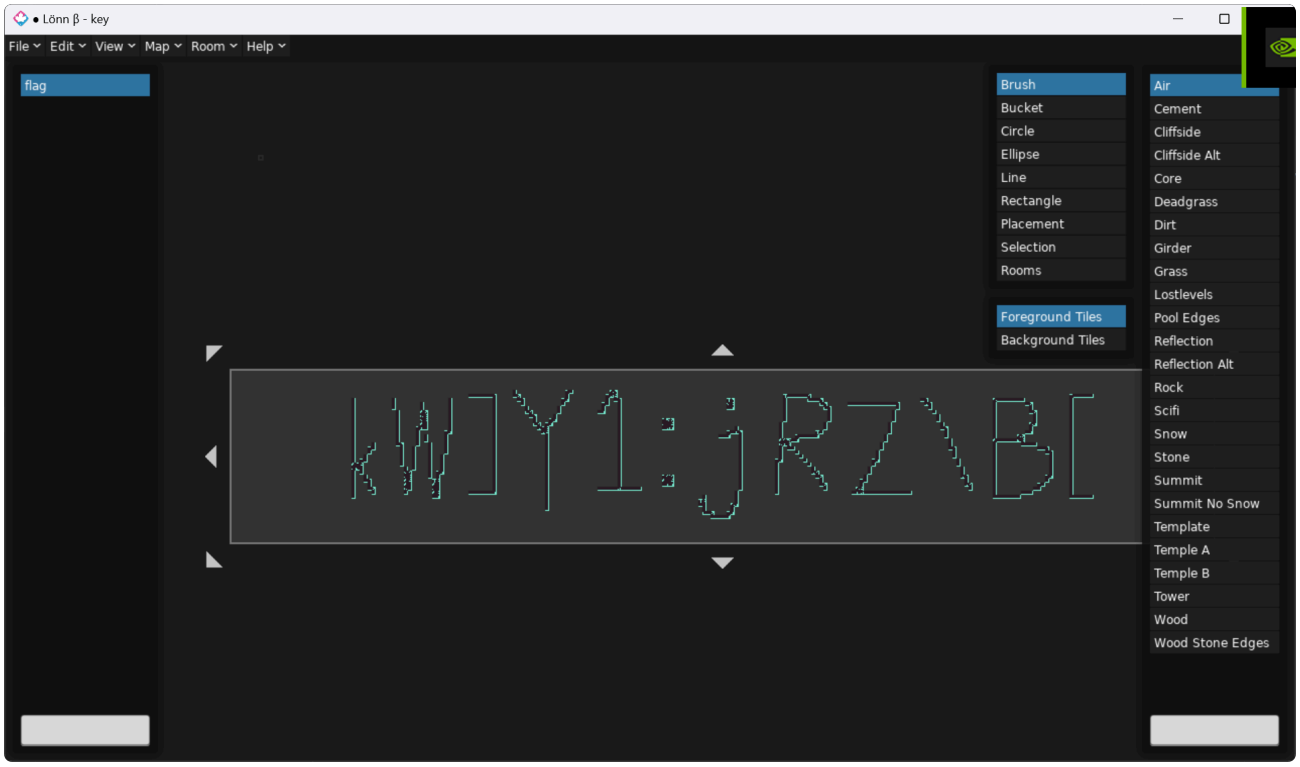
我的评价就是表格解压完后 `media` 里的图



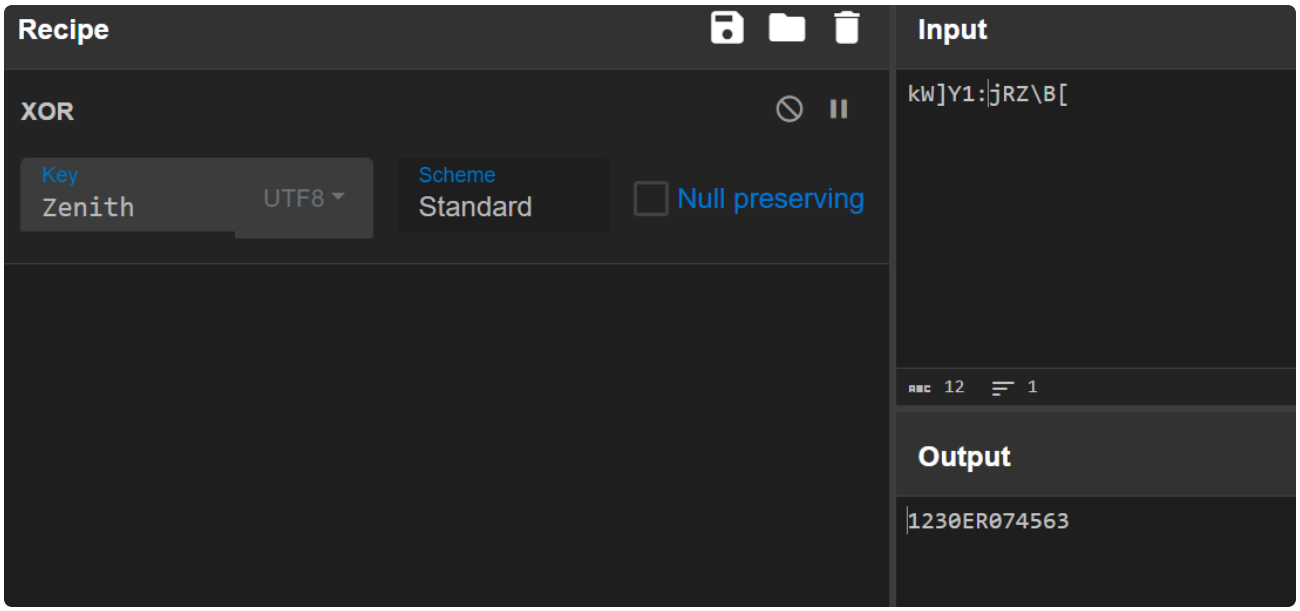
另外感谢出题人让我把刚卸的蔚蓝又下了回来。



用蔚蓝地图编辑软件 `Loenn` 打开异或后的 `key.bin`



`kW]Y1: jRZ\B[`，本来以为这步是经典复刻，结果是来纠错的。



密码是 `1230ER074563`

`LZSDS{Y0u_F1nd_th3_H1dd3n_P0t@t0!}`

Reverse

HelloHarmony

`Eeee` 实现了个移位加密，小写字母还多移了 `2` 位。

```
public Object transform(Object functionObject, Object newTarget, Eeee this, Object arg0, Object arg1) {
    Object obj = "";
    Object obj2 = getiterator(arg0);
    Object obj3 = obj2.next;
    int i = 0;
    while (true) {
        Object callthisN = obj3();
        throw.ifnotobject(callthisN);
        if (istrue(callthisN.done) != null) {
            return obj;
        }
        r28 = callthisN.value;
        try {
            r28 = isfalse((r28 >= "A" ? 1 : 0));
            if (r28 == 0 && isfalse((r28 <= "Z" ? 1 : 0)) == null) {
                obj = (obj == true ? 1 : 0) + this.shiftChar(r28, arg1, "A".charCodeAt(0));
            } else if (isfalse((r28 >= "a" ? 1 : 0)) == null && isfalse((r28 <= "z" ? 1 : 0)) == null) {
                obj = (obj == true ? 1 : 0) + this.shiftChar(r28, arg1 + 2, "a".charCodeAt(0));
            } else {
                obj = (obj == true ? 1 : 0) + r28;
            }
        } catch (ExceptionI0 unused) {
            r14 = r28;
            if (istrue(i) == null) {
                i = 1;
                Object obj4 = null;
                r282 = hole;
                try {
                    Object obj5 = obj2.return;
                    obj3 = obj5;
                    r282 = (0 == obj5 ? 1 : 0);
                } catch (ExceptionI0 unused2) {
                }
                if (r282 == 0) {
                    obj4 = obj3();
                    throw(r14);
                    throw.ifnotobject(obj4);
                }
            }
            throw(r14);
        }
    }
}
```

去 `Index` 找到调用的地方，还加载了个 `libentry.so` ，那么现在就是 `IDA` 时间了。

```
/* JADX WARN: Type inference failed for: r14v13, types: [Object, java.lang.Class] */
public Object #6729484196641124512#(Object functionObject, Object newTarget, Index this) {
    if (("LZSDS" == _lexenv_0_1_.username ? 1 : 0) == 0) {
        AlertDialog.show(createobjectwithbuffer(["title", "Wrong", "message", "用户名错误"]));
        return null;
    }
    Object newobjrange = import { Eeee } from "@normalized:N&&&entry/src/main/ets/Emmm/Eeee&"(5);
    Object testNapi = import { default as testNapi } from "@normalized:Y&&&libentry.so&";
    if ((200 == testNapi.check(newobjrange.encrypt(_lexenv_0_1_.password)) ? 1 : 0) != 0) {
        AlertDialog.show(createobjectwithbuffer(["title", "Gooooooood :)", "message", "恭喜你又又又又登录成功啦"]));
        return null;
    }
    AlertDialog.show(createobjectwithbuffer(["title", "What???", "message", "好像不太对呢..."]));
    return null;
}
```

```
__int64 __fastcall sub_1F94(__int64 a1, __int64 a2)
{
  int v2; // w0
  unsigned __int64 i; // [xsp+28h] [xbp-1B8h]
  __int64 v7; // [xsp+48h] [xbp-198h] BYREF
  __int64 v8; // [xsp+50h] [xbp-190h] BYREF
  __int64 v9; // [xsp+58h] [xbp-188h] BYREF
  unsigned __int8 v10[288]; // [xsp+60h] [xbp-180h] BYREF
  unsigned __int8 v11[40]; // [xsp+180h] [xbp-60h] BYREF
  unsigned __int8 s[40]; // [xsp+1A8h] [xbp-38h] BYREF
  __int64 v13; // [xsp+1D0h] [xbp-10h] BYREF

  v9 = 1LL;
  v13 = 0LL;
  napi_get_cb_info(a1, a2, &v9, &v13, 0LL);
  memset(s, 0, sizeof(s));
  v8 = 0LL;
  napi_get_value_string_utf8(a1, v13, s, 40LL, &v8);
  v2 = strlen(key);
  WTF::WTF((WTF *)v10, (const unsigned __int8 *)key, v2);
  WTF::heiheihei((WTF *)v10, s, v11);
  for ( i = 0LL; i < 0x28; ++i )
  {
    if ( eeeee[i] != v11[i] )
    {
      napi_create_int32(a1, 404LL, &v7);
      return v7;
    }
  }
  napi_create_int32(a1, 200LL, &v7);
  return v7;
}
```

`WTF` 实现数组的初始化，`heiheihei` 进行数据的加密。

```
void __fastcall WTF::WTF(WTF *this, const unsigned __int8 *a2, int a3)
{
  WTF::banana(this);
  WTF::bananana(this, a2, a3);
}
```

`banana` 看起来是建了个盒来混淆。

```
__int64 __fastcall WTF::banana(__int64 this)
{
  int i; // [xsp+14h] [xbp-Ch]

  for ( i = 0; i < 256; ++i )
    *(_BYTE *)(this + i) = (167 * i + 173) % 256;
  return this;
}
```

`bananana` 拿前面的 `key` 进行了密钥的初始化。

```
__int64 __fastcall WTF::bananana(__int64 this, const unsigned __int8 *a2, int a3)
{
  int i; // [xsp+8h] [xbp-18h]

  for ( i = 0; i < 8; ++i )
    *(_DWORD *)(this + 256 + 4LL * i) = (a2[(i + 1) % a3] << 16) | (a2[i % a3] << 24) | (a2[(i + 2) % a3] << 8) | a2[(i + 3) % a3];
  return this;
}
```

`heiheihei` 进行 8 次循环来混淆加密。

```
void *__fastcall WTF::heiheihei(WTF *this, const unsigned __int8 *a2, unsigned __int8 *a3)
{
  void *result; // x0
  int i; // [xsp+14h] [xbp-1Ch]

  result = memcpy(a3, a2, 0x28uLL);
  for ( i = 0; i < 8; ++i )
  {
    WTF::blablablablabla(this, a3, i);
    WTF::bla(this, a3);
    result = (void *)WTF::blablabla(this, a3);
  }
  return result;
}
```

摆了，不想写了，下面是我用 GPT 缝了所有代码的大成之作。

```c
#include <stdio.h>
#include <string.h>
#include <stdint.h>

// 定义结构体来模拟 this 指针
typedef struct {
    unsigned char data[288]; // 288 字节的数组
} WTF;

typedef struct {
    int shift;
} Eeee;

// Eeee 结构体的初始化函数
void Eeee_init (Eeee *self, int arg0) {
```

```
    self->shift = arg0 % 26;
}

// 字符移位函数
char shift_char (char char_to_shift, int shift_amount, int base_char_code)
{
    return (char)(((char_to_shift - base_char_code + shift_amount + 26) %
26) + base_char_code);
}

// 字符串转换函数
void transform (Eeee *self, const char *input_str, char *result, int
shift_amount) {
    int i;
    for (i = 0; input_str[i] != '\0'; i++) {
        char char_to_shift = input_str[i];
        if (char_to_shift >= 'A' && char_to_shift <= 'Z') {
            result[i] = shift_char (char_to_shift, shift_amount, 'A');
        } else if (char_to_shift >= 'a' && char_to_shift <= 'z') {
            result[i] = shift_char (char_to_shift, shift_amount - 2, 'a');
        } else {
            result[i] = char_to_shift;
        }
    }
    result[i] = '\0';
}

// 解密函数
void decrypto (Eeee *self, const char *arg0, char *result) {
    transform (self, arg0, result, -self->shift);
}

// 加密函数
void encrypto (Eeee *self, const char *arg0, char *result) {
    transform (self, arg0, result, self->shift);
}

// 初始化 key
void initialize_key (unsigned int *result) {
    unsigned char a2[] = {'H', 'e', 'l', 'l', 'o', 'S', 'D', 'S'};
    int a3 = sizeof (a2) / sizeof (a2[0]);  // 数组长度
    for (int i = 0; i < 8; ++i) {
        result[i] = a2[(i + 3) % a3] | (a2[(i + 2) % a3] << 8) | (a2[(i +
1) % a3] << 16) | (a2[i % a3] << 24);
    }
}
```

```
// 初始化 WTF 结构体的前 256 个元素
__int64 __fastcall banana (WTF *this) {
    for (int i = 0; i < 256; ++i) {
        this->data[i] = (167 * i + 173) % 256;
    }
    return (__int64) this;
}

// 异或操作
void xor_operation (WTF *this, uint8_t *data, int index) {
    data[0] ^= this->data[256 + 4 * index];
    data[1] ^= this->data[256 + 4 * index];
}

// 替换操作
void substitution (WTF *this, uint8_t *data) {
    for (int i = 0; i < 40; ++i) {
        data[i] = this->data[data[i]];
    }
}

// 循环移位操作
void rotate_left (uint8_t *data) {
    uint8_t temp = data[0];
    for (int i = 0; i < 39; ++i) {
        data[i] = data[i + 1];
    }
    data[39] = temp;
}

// 反向异或操作
void reverse_xor_operation (WTF *this, uint8_t *data, int index, unsigned
int *key) {
    uint32_t var_1 = (data[3] << 24) + (data[2] << 16) + (data[1] << 8) +
data[0];
    var_1 ^= key[index];
    data[0] = var_1 & 0xFF;
    data[1] = (var_1 >> 8) & 0xFF;
    data[2] = (var_1 >> 16) & 0xFF;
    data[3] = (var_1 >> 24) & 0xFF;

    uint32_t var_2 = (data[7] << 24) + (data[6] << 16) + (data[5] << 8) +
data[4];
    var_2 ^= key[index];
    data[4] = var_2 & 0xFF;
```

```
    data[5] = (var_2 >> 8) & 0xFF;
    data[6] = (var_2 >> 16) & 0xFF;
    data[7] = (var_2 >> 24) & 0xFF;
}

// 反向替换操作
void reverse_substitution (WTF *this, uint8_t *data) {
    for (int i = 0; i < 40; ++i) {
        for (int j = 0; j < 256; ++j) {
            if (this->data[j] == data[i]) {
                data[i] = j;
                break;
            }
        }
    }
}

// 反向循环移位操作
void reverse_rotate_left (uint8_t *data) {
    uint8_t temp = data[39];
    for (int i = 39; i > 0; --i) {
        data[i] = data[i - 1];
    }
    data[0] = temp;
}

// 加密函数
void *encrypt (WTF *this, const uint8_t *input, uint8_t *output) {
    memcpy (output, input, 0x28);
    for (int i = 0; i < 8; ++i) {
        xor_operation (this, output, i);
        substitution (this, output);
        rotate_left (output);
    }
    return output;
}

// 解密函数
void *decrypt (WTF *this, const uint8_t *input, uint8_t *output, unsigned
int *key) {
    memcpy (output, input, 0x28);
    for (int i = 7; i >= 0; --i) {
        reverse_rotate_left (output);
        reverse_substitution (this, output);
        reverse_xor_operation (this, output, i, key);
    }
```

```
    return output;
}

// 辅助函数：打印 WTF 结构体中的值
void print_wtf (const WTF *wtf) {
    printf ("WTF. data = {\n");
    for (int i = 0; i < 256; ++i) {
        printf ("0x%02X", wtf->data[i]);
        if (i < 287) {
            printf (", ");
        }
        if ((i + 1) % 16 == 0) {
            printf ("\n");
        }
    }
    printf ("}\n");
}

int main () {
    // 初始化 key
    unsigned int key[8];
    initialize_key (key);

    // 初始化WTF结构体
    WTF wtf;
    memset(wtf.data, 0, sizeof(wtf.data));

    // 调用banana函数进行前256个元素的初始化
    banana(&wtf);

    // 打印结果
    print_wtf(&wtf);

    // 加密和解密示例
    uint8_t encrypted[40] = {0xF6, 0xB0, 0xA6, 0x36, 0x9A, 0xB3, 0x2B,
0xBF, 0x94, 0x54, 0x15, 0x97, 0x93, 0x59, 0xBF, 0x50, 0x4D, 0xBF, 0x0A,
0x59, 0x06, 0xD7, 0x97, 0x50, 0xD6, 0x59, 0x54, 0xD7, 0xCF, 0x06, 0x5D,
0x20, 0x1D, 0x5A, 0x22, 0xEE, 0x99, 0x1F, 0xE1, 0x18};
    uint8_t decrypted[40] = {};

    decrypt(&wtf, encrypted, decrypted, key);

    printf("Decrypted: %s\n", decrypted);
    Eeee eeee;
    Eeee_init(&eeee, 5);  // Example shift value
```

```c
    char flag[100];
    decrypto(&eeee, decrypted, flag);
    printf("flag: %s\n", flag);

    return 0;
}
```

Super Panda Girl

Unity 3D mono，dnspy 打开 Assembly-CSharp.dll 就是源代码了。

```csharp
private byte[] ssS5sss5s(string sSs5s5s5, string S5SSS5s5)
{
    int length = S5SSS5s5.Length;
    int length2 = sSs5s5s5.Length;
    byte[] bytes = Encoding.UTF8.GetBytes(sSs5s5s5);
    byte[] array = new byte[256];
    byte[] array2 = new byte[length2];
    for (int i = 0; i < 256; i++)
    {
        array[i] = (byte)i;
    }
    int num = 0;
    for (int j = 0; j < 256; j++)
    {
        num = (num + (int)array[j] + (int)S5SSS5s5[j % length]) % 256;
        byte b = array[j];
        array[j] = array[num];
        array[num] = b;
    }
    int num2 = 0;
    int num3 = 0;
    for (int k = 0; k < length2; k++)
    {
        num2 = (num2 + 1) % 256;
        num3 = (num3 + (int)array[num2]) % 256;
        byte b2 = array[num2];
        array[num2] = array[num3];
        array[num3] = b2;
        byte b3 = (byte)((int)(array[num2] + array[num3]) % 256);
        array2[k] = bytes[k] ^ b3;
    }
    return array2;
}
```

你这 RC4 有问题啊，直接和你爆了！
但在爆之前看看前面有没有别的东西

```
// Token: 0x06000019 RID: 25 RVA: 0x00002764 File Offset: 0x00000964
public void VerifyPassword()
{
    string text = this.inputField.text;
    if (!this.pandaKey.ssSSs5s5s5(text))
    {
        this.ShowIncorrectPasswordUI();
        return;
    }
    string text2 = this.code(this.GetEvenCharacters(text));
    this.ShowCorrectPasswordUI("LZSDS{" + text2 + "}");
}
```

```
// Token: 0x0600001D RID: 29 RVA: 0x00002828 File Offset: 0x00000A28
private string GetEvenCharacters(string input)
{
    StringBuilder stringBuilder = new StringBuilder();
    for (int i = 0; i < input.Length; i += 2)
    {
        stringBuilder.Append(input[i]);
    }
    return stringBuilder.ToString();
}
```

```
// Token: 0x0600001E RID: 30 RVA: 0x00002860 File Offset: 0x00000A60
private string code(string text)
{
    return Convert.ToBase64String(Encoding.UTF8.GetBytes(text));
}
```

只取奇数位字符，然后 `base64` 编码后就是 `flag` 了，那直接爆了。

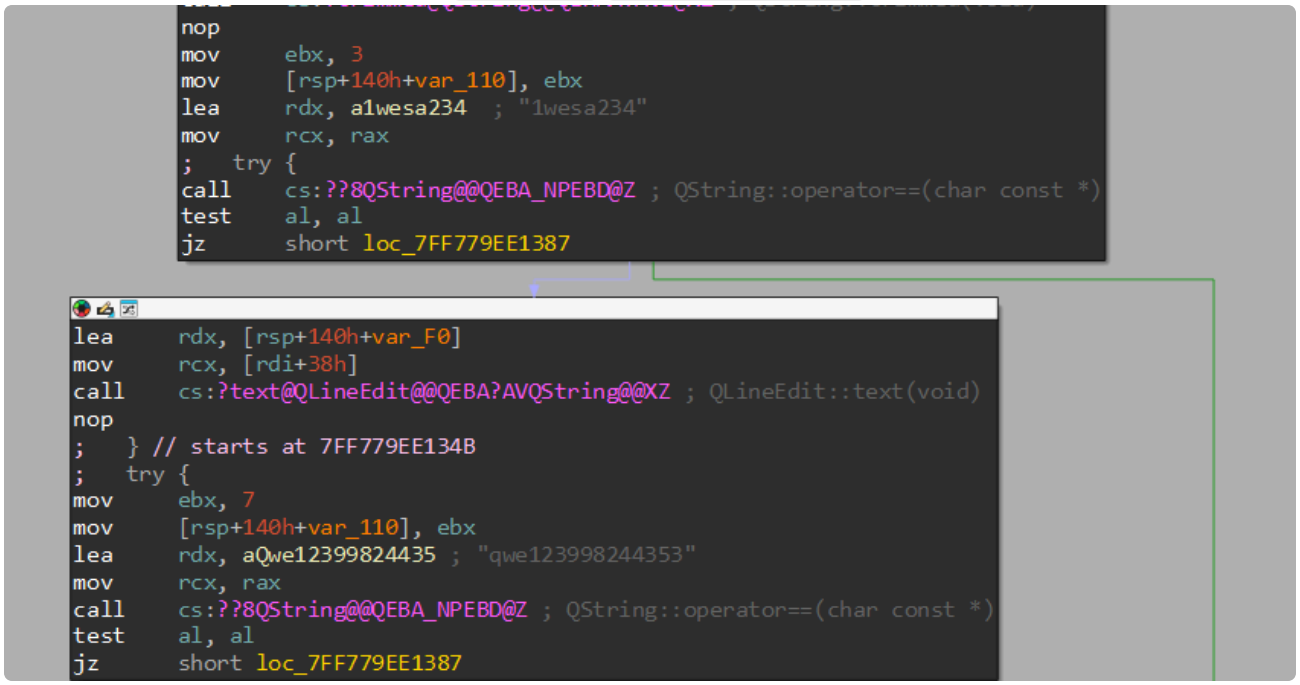
```python
import string
import base64


def ssS5sss5s(sSs5s5s5, S5SSS5s5):
    length = len(S5SSS5s5)
    length2 = len(sSs5s5s5)
    bytes_data = sSs5s5s5.encode('utf-8')
    S = list(range(256))
    array2 = [0] * length2

    num = 0
    for j in range(256):
        num = (num + S[j] + ord(S5SSS5s5[j % length])) % 256
        S[j], S[num] = S[num], S[j]
```

```python
    i = 0
    j = 0
    for k in range(length2):
        i = (i + 1) % 256
        j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i]

        b3 = (S[i] + S[j]) % 256

        array2[k] = bytes_data[k] ^ b3

    return bytes(array2)


def brute_force_decrypt(encrypted_data, key):
    possible_chars = string.printable
    decrypted_data = []

    for i in range(len(encrypted_data)):
        for char in possible_chars:
            # 构建当前字符的明文
            current_plaintext = ''.join(decrypted_data) + char
            # 使用 ssS5sss5s 加密当前明文
            encrypted_result = ssS5sss5s(current_plaintext, key)
            # 检查加密结果是否与密文匹配
            if encrypted_result[i] == encrypted_data[i]:
                decrypted_data.append(char)
                break

    return ''.join(decrypted_data)


def get_even_characters(input_str):
    result = []
    for i in range(0, len(input_str), 2):
        result.append(input_str[i])
    return ''.join(result)


# 已知的密文和密钥
encrypted_data = bytes([57, 244, 117, 200, 213, 87, 194, 195, 164, 100,
103, 63, 19, 79, 137, 70, 201, 24, 163, 129, 237, 210, 5, 19, 35, 21])
key = "LZSDS"

# 逐字节爆破解密
```

```python
decrypted_data = brute_force_decrypt(encrypted_data, key)
print("Decrypted Data:", decrypted_data)

input_str = decrypted_data
even_chars = get_even_characters(input_str)
print(even_chars)

print(base64.b64encode(even_chars.encode()).decode())
```

Potato Toolkit

一进去就看到两个字符串比较，`1wesa234` `qwe123998244353`



但输了这两个字符串就会一路直奔到 `exit (0);`

```
53  if ( v7 )
54  {
55    for ( i = 0; i <= 100; ++i )
56    {
57      QProgressBar::setValue(a1[12], i);
58      sleep(0);
59    }
60    v20[0] = _mm_load_si128(&xmmword_7FF779EE6870);
61    v20[1] = _mm_load_si128(&xmmword_7FF779EE68A0);
62    v20[2] = _mm_load_si128(&xmmword_7FF779EE6840);
63    v20[3] = _mm_load_si128(&xmmword_7FF779EE6890);
64    v20[4] = _mm_load_si128(&xmmword_7FF779EE6850);
65    v20[5] = _mm_load_si128(&xmmword_7FF779EE68B0);
66    v20[6] = _mm_load_si128(&xmmword_7FF779EE6860);
67    v20[7] = _mm_load_si128(&xmmword_7FF779EE6880);
68    v20[8] = _mm_load_si128(&xmmword_7FF779EE6830);
69    v21 = 24;
70    v22 = 79;
71    QString::QString(v18, byte_7FF779EE6800);
72    v9 = QLineEdit::text(a1[6], v19);
73    QString::trimmed(v9, v17);
74    QString::~QString(v19);
75    v10 = v20;
76    v11 = 38LL;
77    do
78    {
79      v12 = QString::length(v17);
80      v13 = QString::operator[](v17, v2 % v12);
81      v14 = QChar::unicode(v13);
82      v15 = QChar::QChar(&v16, *v10 ^ *v14);
83      QString::operator+=(v18, *v15);
84      ++v2;
85      v10 += 4;
86      --v11;
87    }
88    while ( v11 );
      exit(0);
```

看不懂 QT 函数，开个动调慢慢看，发现到 83 行的时候 v16 就是 flag 的值，一个一个读几十个字符总比我看逆向写脚本快。

`LZSDS{@_v3ry_very_Ab3tr@ct_P0t@to_Guy}`

Crypto

xorsa

e 和 phi 不互素

```
from Crypto.Util.number import *
from gmpy2 import *
# import uuid
# flag='LZSDS{'+str(uuid.uuid4())+'}'

# while True:
#     p=getPrime(512)
#     if p.bit_length()==512:
#         break
# mask="????"
# q=p^mask
# hint1=p^q
# hint2=q
# n=p*q
# e=2026
# m=bytes_to_long(flag.encode())
# c=pow(m,e,n)
# print("c =",c)
# print("n =",n)
# print("hint1 =",hint1)
# print("hint2 =",hint2)

# '''
c =
13760578729891127041098229431259961120216468948795732373975536417751222443
06980577569384556000588198162220208988386639557715470122904624588228212705
49691142103071751165741784288230438170419562075032992207210421365158639796
55578210499512044917781566303947681251248645504273995402630701480590505840
473412765662
n =
14247038211821385209759067256846232227444163173099199085257790370590450749
66520655616336475426918225535808494835434582789898723475666213397463311706
29023708118554666653517840271253331126630750853956765011217597866997201490
98576433141817737564928779420725539793335830274229206316999461309927000523
188222801659
hint1 =
89385386199617313997160166654705640849862438803949289184823742958145093533
82364651201249532111268951793354572124324033902502588541297713297622432670
722730
```

```
hint2 =
14932981552434748373200928493257503877595196438793886092083144940006055540
20636706320849032906759121914762492378489852575583260177546578935320977613
050647
# '''

q = hint2
p = hint1 ^ q
phi = (p - 1) * (q - 1)

e = 2026
t = gmpy2.gcd(e, phi)
t1 = e // t
dt1 = gmpy2.invert(t1, phi)
mt1 = pow(c, dt1, n)
print(mt1)
s, m = gmpy2.iroot(mt1, t)
print(long_to_bytes((s)))
```